# Práctica de Laboratorio 3

Freddy Ovalle C.I 31709650 and Juan Bermúdez C.I 31608369

Universidad de Carabobo

Facultad de Ciencias y Tecnología
Departamento de Computación

13 de Octubre de 2026

## Objetivo

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a operaciones de entrada/salida (E/S) mediante el manejo de dispositivos de hardware mapeados en memoria.

## 1 Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema de Memory-Mapped I/O (E/S mapeada en memoria), la memoria y los periféricos comparten un único espacio de direcciones físico. Los registros de control y datos de los dispositivos se asignan a rangos específicos de memoria.

Al inicio, se asignan rangos de direcciones únicos a cada dispositivo. Cuando la CPU escribe en una dirección "mapeada", la información va al dispositivo en lugar de a la RAM. Comunmente los dispositivos se suelen mapear en la región alta del espacio de direcciones, además el memory-mapped al cargar en memoria los dispositivos, nos permite que la CPU use instrucciones estándar de carga/almacenamiento (lw/sw) para comunicarse con ellos como si fueran celdas de memoria RAM, simplificando la gestión.

## 2 ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

La principal diferencia entre ambas, es la manera en como se accede a la memoria, mientras que el memory-mapped I/O hace uso de las instrucciones simples de carga y almacenamiento, el entrada/salida por puertos depende de instrucciones especiales que no siempre son iguales en todas las arquitecturas.

La desventaja del memory-mapped es el costo en memoria RAM que implica el uso de este manejo, cada registro tiene que ocupar un espacio distinto en memoria RAM y su ventaja consiste en la simplificación del diseño, no debemos utilizar instrucciones especiales, facilitando la estructura.

Por otra parte la entrada/salida por puertos separa las direcciones de memoria y la de dispositivos,sin ocupar la memoria RAM y nos permite un mejor control de las operaciones entrada/salida, a costo de esto su desventaja es que requiere un diseño mas complejo de instrucciones del procesador y genera mas restricciones ya que no podemos trabajar con el esquema simple de carga/almacenamiento.

MIPS32 hace uso principal del memory-mapped ya que este coincide con la arquitectura simple del entorno MIPS32, permitiendonos hacer uso de instrucciones ya incluidas en este, como lw y sw, siendo comodo de trabajar y mas sencillo.

## 3 En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos comparten la misma dirección o un rango de direcciones superpuesto, pueden surgir varios problemas principalmente cuando el procesador accede a esa dirección, ambos dispositivos podrían responder al mismo tiempo, podría recibir datos mezclados o provenientes del dispositivo equivocado, una instrucción podría modificar el estado de dos dispositivos simultáneamente e incluso un comportamiento impredecible del sistema, ya que el hardware no sabría cuál dispositivo debe responder a la operación.

Este modelo no es compatible con el solapamiento, la manera de evitar el conflicto es asegurarnos de la correcta asignación de los dispositivos con diferentes espacios de memoria a cada uno, de esta manera no habrá conflictos en mezcla de datos , entradas o salidas entre dispositivos.

## 4 ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

El diseño de el memory-mapped utiliza las instrucciones estandar aplicadas para la carga y almacenamiento en memoria RAM, y al manejar los dispositivos desde la memoria RAM, podemos interactuar con ellos con el uso de instrucciones LOAD/STORE estandar, por lo que no tendremos que hacer uso de una estructura nueva sino que los mismos conceptos, volviendo mas simple el manejo.

En el manejo E/S por puertos se utilizan instrucciones adicionales para la entrada/salida de información, como por ejemplo IN/OUT en algunas arquitecturas.

## 5 ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador ejecuta una instrucción de acceso a memoria (por ejemplo lw o sw): El procesador coloca la dirección de memoria en el bus de direcciones y la señal de control indica si la operación es lectura o escritura. Si es una lectura, el dispositivo correspondiente coloca el dato en el bus de datos para que el procesador lo lea, si es una escritura, el procesador coloca el dato en el bus de datos y el dispositivo lo recibe.

El hardware identifica que debe acceder a un periférico mediante un circuito de decodificación de direcciones. Este circuito analiza la dirección enviada por el procesador y determina a qué componente corresponde y activa dicho componente, si es el rango asignado a la memoria RAM, o si es el rango de un dispositivo E/S, asi nos aseguramos que solo el componente cuyo rango de direcciones coincide con la dirección enviada por el procesador responde a la operación.

## 6 ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En un sistema que utiliza Memory-mapped I/O, los dispositivos están asignados a direcciones dentro del mismo espacio de memoria que la RAM. Por ello, desde el punto de vista del procesador, el acceso a un dispositivo se realiza mediante instrucciones normales de memoria como lw o sw.

Por esta razón, un programa normal podría acceder a un dispositivo si intenta leer o escribir en la dirección de memoria donde está mapeado, esto podría provocar modificaciones incorrectas en el hardware o interferir con el funcionamiento del sistema.

Para evitar accesos no autorizados a dispositivos mapeados los sistemas utilizan varios mecanismos de protección:

### 6.1 Modos de ejecución del procesador

El procesador distingue entre modo usuario y modo kernel. Solo el sistema operativo en modo kernel puede acceder directamente a ciertas regiones de memoria donde se encuentran los dispositivos.

### 6.2 Protección de memoria mediante la MMU

: Controla qué direcciones de memoria puede acceder cada proceso y genera una excepción si intenta acceder a una región restringida.

### 6.3 Permisos en tablas de páginas

El sistema operativo define permisos de lectura, escritura o ejecución para cada región de memoria, impidiendo que programas sin privilegios accedan a direcciones reservadas para dispositivos.

## 7 ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Para evitar esperas activas innecesarias al interactuar con dispositivos en sistemas con Memory-mapped I/O, se pueden emplear las siguientes técnicas:

### 7.1 Interrupciones

El dispositivo envía una señal de interrupción al procesador cuando la operación termina o cuando los datos están disponibles. Así, el procesador no necesita verificar constantemente el estado del dispositivo y puede ejecutar otras tareas.

### 7.2 Acceso Directo a Memoria (DMA)

Mediante el acceso directo a memoria o DMA, un controlador especializado se encarga de transferir datos entre el dispositivo y la memoria sin intervención continua del procesador. El CPU solo inicia la operación y es notificado cuando finaliza.

### 7.3 Suspensión o bloqueo del proceso

El sistema operativo puede suspender el proceso que está esperando la operación de entrada/salida y reanudarlo cuando el dispositivo esté listo, permitiendo que el procesador ejecute otros procesos mientras tanto.

# 8 Análisis y Discusión de los Resultados

Al observar nuestros algoritmos podemos notar que presentan estructuras similares, pero con el uso de diferentes dispositivos, en todos los casos presentan una notacion Big-O de O(1), ya que ninguna operación interna depende de las entradas, solamente dependerá de cuanto tiempo tarde en cambiar de estado ya sea -1 en error, 1 en actividad y 0 en reposo en el caso de los dos primeros algoritmos (Luminosidad, Presion) o solamente 0 en reposo y 1 en actividad en caso del tercero.

Al usar MIPS32 como procesador de estos algoritmos solamente podemos evaluar ejecuciones hipoteticas, ya que al no existir un hardware real de medición los datos que proporciona este los ingresamos nosotros mismos, y al indicar el cambio de estado se hace mediante la ejecución paso a paso en el apartado de Data segment, de esta manera podemos ver el comportamiento del computador mientras espera la señal de actividad o en dado caso de error.

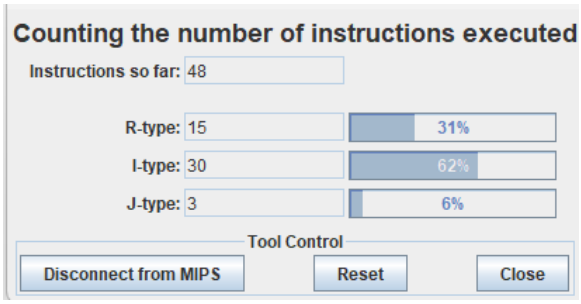



Figure 1: INSTRUCCIONES ALGORITMO DE LUMINOSIDAD EN ESTADO 1 AL INICIO

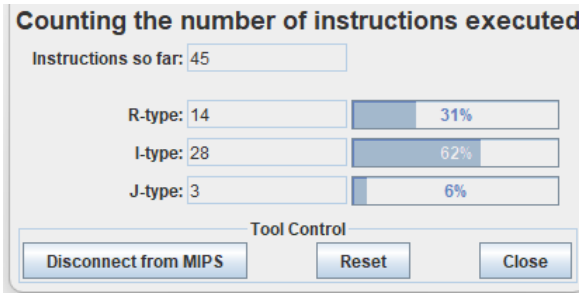



Figure 2: INSTRUCCIONES ALGORITMO DE PRESION EN ESTADO 1 AL INICIO

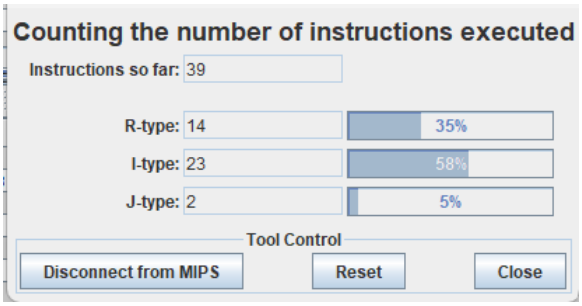



Figure 3: INSTRUCCIONES ALGORITMO DE TENSION EN ESTADO 1 AL INICIO

En un escenario real, gracias a la compatibilidad del entorno MIPS32 con el memmory-mapped, se podrian implementar estos algoritmos e interactuar con los respectivos dispositivos desde las direcciones de memoria asignadas, en este caso se obtendrian los datos a partir de dichas direcciones y el cambio de estado sincronizado con el real.